

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead



UNIT IV · SECURITY & DATA ANALYTICS

Topic 1

10 MIN

Why IoT Security Matters

Real breaches, and the map for everything this unit covers

- Real breach examples
- Requirements to threats
- Analytics & blockchain

Every Sensor Is Also a Door

~2 min

Everything Unit III built — devices, edge, cloud, AI — is only useful if it can be trusted.

In Simple Words

A laptop gets security updates for years. A ten-dollar sensor often ships once and is never patched again.

Multiply that by billions of connected devices, each a tiny, often-forgotten door into a home, a hospital or a factory network.

Why IoT is a harder security problem

- Tiny devices, little power for encryption
- Deployed for years without updates
- Huge scale — billions of endpoints

What Happens When It Fails

A device becomes a weapon

Compromised cameras and routers were hijacked to attack other targets at massive scale.

Physical systems get controlled

A hacked sensor or actuator does not just leak data — it can move a car or a valve.

One weak device exposes a network

The smallest, most trivial-looking gadget is often the easiest way in.

The next slide looks at three real breaches that made exactly this point.

1 Real-World Breach Examples

~4 min

Three incidents that turned "just a sensor" into a serious lesson.

Mirai Botnet (2016)

Malware scanned the internet for cameras and routers still using their factory-default password, then hijacked hundreds of thousands of them to flood major websites offline in one of the largest DDoS attacks ever recorded.

Jeep Cherokee Hack (2015)

Security researchers remotely took control of a moving Jeep's steering, brakes and transmission through its cellular-connected infotainment system — with no physical access to the vehicle.

Casino Fish-Tank Thermometer

Attackers entered a casino's network through an internet-connected fish-tank thermometer, then moved sideways to steal its high-roller customer database.

A weak password, a connected car, and a fish-tank thermometer — the device barely matters; the connection does.

2 Framing This Unit

~2 min

Security requirements, threats, analytics and blockchain — four ideas, one thread.

1

Requirements

What "secure" actually means for a device

→

2

Threats

How those requirements get broken

→

3

Analytics

Using data to catch problems early

→

4

Blockchain

A different way to establish trust

The logic behind the order

You cannot spot a threat without first knowing what "secure" was supposed to look like. Once threats are understood, analytics is how a system notices when something has gone wrong — the same anomaly-detection idea from Unit III, now aimed at attacks instead of failures. Blockchain closes the unit as an alternative way to establish trust, without relying on one central authority at all.

Four topics, one throughline: know the rules before you can see them being broken.

The Unit IV roadmap

~1 min

6 topics, 180 minutes.



1
Introduction

You are here

2
Security Fundamentals

3
Threats & Attacks

4
Data Analytics

5
Blockchain + IoT

6
Recap & Assessment

You are here: Topic 1. Data Analytics carries the most depth at 45 minutes.

One-line takeaway

> *“Every device you connect is a door you now have to guard — Mirai, the Jeep hack and the fish-tank thermometer all made that lesson the hard way.”*

Breach examples | Requirements | Threats | Analytics | Blockchain

Up next: Topic 2 — IoT Security Fundamentals: requirements, authentication, encryption and secure communication.



Security Is Built in Layers

~3 min

No single control makes a device secure. Each of the eight ideas today covers one layer of the same wall.

In Simple Words

Security is not one switch you flip on. It is a series of questions asked at every stage: who are you, what are you allowed to do, and can anyone else read or tamper with what you send?

Today walks through those questions in order, then applies all of them to the device, the network and the cloud.

Today, in 40 minutes

8 ideas — from what "secure" requires, to defending three layers at once.

Eight ideas, four pairs

Requirements

What "secure" has to mean

Authentication + Authorization

Prove who you are, then what you may do

Encryption + Secure comms

Protect data at rest and moving between devices

Device, network & cloud security

Apply all of it at every layer of the system

Each pair builds on the one before it — skipping ahead usually means missing the "why."

1 Security Requirements

~4 min

The CIA triad, plus two ideas IoT adds on top of it.

Confidentiality

Only the intended reader can see the data.

Integrity

Data is not altered in transit or at rest, unnoticed.

Availability

The system keeps working when it is needed.

Authenticity

A message really came from the device it claims to.

Non-repudiation

A sender cannot later deny having sent it.

The first three are the classic CIA triad; the last two matter especially once devices act on data automatically.

2 Authentication

~3 min

Proving a device or user really is who it claims to be.

The one question it answers

"Are you really who you say you are?" A gateway accepting readings from a thousand sensors needs to know each one is genuine, not a spoofed impostor sending fake data.

Common factors

- Something you know — a password or PIN
- Something you have — a certificate or key
- Something you are — biometrics, rarely used on tiny devices

How devices actually authenticate

1. **Pre-shared keys**

   A secret baked in at manufacture time, simple but hard to rotate

2. **Digital certificates**

   A signed identity a device presents during connection, widely used with TLS

3. **Tokens**

   A short-lived credential issued after an initial login, common for cloud APIs

Weak or default passwords, as in the Mirai botnet, defeat authentication before any other control gets a chance.

3 Authorization

~3 min

Once identity is confirmed, what is this device or user actually allowed to do?

The idea

Least privilege

Give every device and user the minimum access needed, nothing more

Role-based access

A temperature sensor can send readings; it should never be able to change firmware

Scoped permissions

An app that reads a thermostat should not automatically control the door lock

Authentication vs Authorization

	AUTHN	AUTHZ
Question	Who are you?	What can you do?
Happens	First, at connection	After identity is known
Example	Certificate accepted	Read-only sensor access

A device can be perfectly authenticated and still be given far too much authorization — both checks are needed.

4 Encryption

Making data unreadable to anyone without the right key.

~4 min

Symmetric

One shared key

Same key encrypts and decrypts the data

Fast & lightweight

Cheap enough for a small microcontroller to run

Key distribution is the hard part

Both sides must already share the secret safely

Asymmetric

A public & private key pair

Encrypt with one, only the matching key decrypts

Solves key distribution

The public key can be shared openly, no secret exchange needed

Heavier to compute

Often used just to exchange a lightweight symmetric key

Most real systems use both together: asymmetric to safely hand over a key, symmetric to encrypt the actual traffic.

5 Secure Communication

~4 min

Turning an open wireless link into a protected channel.

Device connects

A sensor initiates contact with a gateway or server

→

Certificates exchanged

Each side proves its identity before anything else happens

→

Session key agreed

A temporary symmetric key is negotiated for this session only

→

Encrypted channel

All further traffic is unreadable to anyone listening in

The protocols behind this handshake

TLS secures connections built on TCP, such as MQTT and HTTPS from Unit II. DTLS provides the same protection for connectionless protocols like CoAP, which run over UDP. Both follow the same handshake pattern shown above — only the underlying transport differs.

This is the same TLS handshake your browser runs every time you visit an https:// site.

What weak security actually costs

~2 min

Minutes

to compromise a default-password device

automated scanners find it fast

Thousands

of devices in a single botnet

as seen with Mirai in 2016

One

weak device is often enough

to reach an entire network

Why fundamentals, not extras

Every breach in Topic 1 traced back to one of the ideas on this slide being skipped or done poorly: Mirai exploited authentication, the Jeep hack exploited network boundaries, the fish-tank thermometer exploited a lack of network segmentation. None of these attacks needed anything exotic — they needed one fundamental left undone. That is exactly why the rest of this unit builds on getting these right first.

Next: applying all of this to the device itself, the network, and the cloud.

6 Device Security

~4 min

Protecting the physical hardware itself, from power-on to end of life.



Hardware root of trust

A tamper-resistant chip stores keys that never leave the hardware.



Secure boot

The device only runs firmware that is cryptographically signed and verified.



Firmware updates

A signed, authenticated path to patch vulnerabilities after deployment.



Tamper resistance

Physical protections that detect or resist someone opening the device.

A device that cannot be updated is a device whose vulnerabilities are permanent — secure boot and signed updates fix that.

7 Network Security

Controlling how devices and traffic move once they are on the network.

~3 min

Core controls

Segmentation

IoT devices sit on their own network segment, separate from sensitive systems

Firewalls & gateways

Filter and inspect traffic entering or leaving the IoT segment

VPN tunnels

Encrypt traffic travelling between a site and the cloud

Monitoring

Watch for the unusual traffic patterns Unit III's anomaly detection can catch

The lesson from the fish tank

The casino's thermometer was never meant to reach the customer database — the network let it. Segmentation means that even if one weak device is compromised, an attacker still hits a wall trying to reach anything else. Treat every device as if it might one day be compromised, and design the network so that alone is not enough.

Network security assumes a device will eventually fail — the goal is to contain the damage, not just prevent it.

8 Cloud Security

~3 min

The device and network can be perfect — the cloud still has to hold up its end.



Identity & access

Strict IAM policies decide exactly who and what can touch stored data.



Encrypted storage

Data at rest is encrypted, not just data moving over the network.



API security

Every API endpoint is authenticated, rate-limited and validated.



Logging & monitoring

Every access is logged, so unusual activity can be traced and caught.

The cloud stores data from every device at once — a single misconfigured bucket can undo every other layer's work.

Defense in Depth

~2 min

All eight ideas today, mapped onto three layers.

Device

Root of trust, secure boot, signed firmware updates

Authentication + Encryption apply here first

Network

Segmentation, firewalls, gateways, VPN tunnels

Secure comms + Authorization control what crosses it

Cloud

IAM, encrypted storage, API security, logging

Every requirement from slide 1 applies here too

An attacker only needs one gap; defense in depth means every layer still has to be broken separately.

The Unit IV roadmap

~1 min

6 topics, 180 minutes.



1 Introduction

2 Security Fundamentals

You are here

3 Threats & Attacks

4 Data Analytics

5 Blockchain + IoT

6 Recap & Assessment

You are here: Topic 2. Eight numbered sections across 10 content slides reflect this topic's 40 minutes.

By the end of this topic

You will be able to:



List

The five security requirements IoT systems need, beyond the classic CIA triad.



Distinguish

Authentication from authorization, and symmetric from asymmetric encryption.



Explain

How a secure communication channel is established, and which protocols do it.



Apply

Defense-in-depth thinking across the device, network and cloud layers.

Look at any IoT deployment and ask: which of these eight ideas would break first?

The big picture

~2 min

Requirements	Confidentiality, integrity, availability, authenticity, non-repudiation — what "secure" has to mean.
Authn / Authz	Prove identity first, then decide what that identity is allowed to do.
Encryption	Symmetric for speed, asymmetric to safely share the key — most systems use both.
Secure comms	A handshake exchanges certificates and agrees a session key before any data flows.
Device / network / cloud	The same requirements, applied at every layer — defense in depth.
Up next	Topic 3: IoT Threats & Attacks — what happens when one of these controls is missing.

One-line takeaway

> *“Security is not a feature you add once — it is eight questions, asked again at the device, the network and the cloud.”*

Requirements | Authentication | Authorization | Encryption | Secure comms | Defense in depth

Up next: Topic 3 — IoT Threats & Attacks: device attacks, eavesdropping, spoofing, DoS, malware and data leakage.



From Fundamentals to Failure Modes

~2 min

Every control from Topic 2 has a matching attack that targets it directly. This topic names six of them.

In Simple Words

Knowing the defenses is only half the picture. To actually secure a system, you have to think like the person trying to break it.

Each threat here targets one specific gap: a device left open, a link left unencrypted, an identity left unverified.

Already seen one of these

The Mirai botnet from Topic 1 was a device attack that led straight into a DoS attack — two of today's six categories, back to back.

Six threats, in order

1. Device attacks
2. Eavesdropping
3. Spoofing
4. Denial-of-Service
5. Malware
6. Data leakage

Same order as the docx outline — each builds a slightly bigger picture of "what goes wrong."

1 Device Attacks

Getting at the hardware itself, often with physical access.

~4 min

The idea

Unlike a server locked in a data center, an IoT device is often sitting in a public hallway, a field, or someone's living room — within physical reach of an attacker.

Common forms

- Tampering — physically opening or modifying a unit
- Firmware extraction via an exposed debug port
- Counterfeit or cloned devices on the network

Why it matters more here

One extracted key rarely stays local. Many deployments reuse the same key or credential across thousands of identical units, so breaking one physical device can hand an attacker the keys to the entire fleet — exactly what secure boot and a hardware root of trust from Topic 2 exist to prevent.

Physical access is often treated as "game over" for security — which is exactly why device security exists.

2 Eavesdropping

~3 min

Silently listening to traffic the sender never meant to share.

Passive by nature

The attacker does not touch the device or the network — just listens to what is already being broadcast.

Wireless is exposed by default

Any radio signal within range can be captured; the question is only whether it can be read.

Replay is the follow-up move

A captured, unencrypted credential can simply be resent later to impersonate the original sender.

The direct defense

Encryption and secure communication from Topic 2 turn a readable broadcast into meaningless noise for anyone without the session key.

Unlike most attacks, eavesdropping leaves no trace — the first sign of it is often the next attack it enables.

3 Spoofing

~3 min

Pretending to be a device, gateway, or location that you are not.

Fake sensor

An impostor device sends false readings that look legitimate to the receiving system.

Rogue gateway

A fake access point tricks nearby devices into connecting to it instead of the real one.

GPS spoofing

Fake satellite signals feed a device false location data, misleading tracking or navigation.

The direct defense

Spoofing targets authentication directly — a device that must prove its identity with a certificate cannot be faked with a copied name alone.

If a system trusts a device just because it says the right name, spoofing is only a matter of time.

4 Denial-of-Service (DoS) Attacks

~4 min

Making a system unavailable, without needing to break in at all.



Flooding

Overwhelming a device or gateway with more traffic than it can process, so real requests get dropped.



Sleep deprivation

Repeatedly waking a battery-powered sensor so it can never rest, draining it far ahead of schedule.



Jamming

Broadcasting noise on the same radio frequency a device relies on, blocking it from communicating at all.



Distributed DoS (DDoS)

Thousands of hijacked devices, like a Mirai botnet, flood a single target all at once.

Availability is one of the five requirements from Topic 2 — DoS attacks are the one category that targets it directly.

Why these six categories cover so much ground

~2 min

Billions

of connected devices exposed

each one a potential entry point

Hundreds of thousands

of devices in one botnet

as with Mirai's DDoS attack

Seconds

for a passive listener to capture

an unencrypted credential in transit

The pattern underneath all six

Device attacks, eavesdropping, spoofing and DoS each target one control from Topic 2 almost one-to-one. The next two — malware and data leakage — are different: they are often what an attacker does after one of the first four already succeeded, spreading the damage further or extracting what they came for.

This is exactly why the last two categories are covered next.

5 Malware

~4 min

Malicious code that infects a device and spreads on its own.

Scan

Malware sweeps the internet for devices with default or weak passwords

→

Infect

It logs in and installs itself, often with no visible sign to the owner

→

Spread

The infected device starts scanning for its next target, worm-style

→

Act

The whole botnet is remotely commanded, e.g. into a DDoS attack

This is the Mirai story, generalized

Mirai from Topic 1 followed exactly this scan-infect-spread-act pattern. What made it dangerous was not a sophisticated exploit — it was that so many devices shared the same easily guessed default password, so one working attack could scale to hundreds of thousands of devices with almost no extra effort.

A device once infected rarely shows it — the first visible sign is often the attack it launches on something else.

6 Data Leakage

~3 min

Sensitive data ending up somewhere it was never meant to go.

Cloud misconfiguration

A storage bucket or database left open to the internet by mistake, no attack needed.

Insecure APIs

An endpoint that returns more data than it should, or skips checking who is asking.

Metadata leakage

Even encrypted traffic can reveal when a device is active, home occupancy or routines.

The casino, again

The fish-tank thermometer breach from Topic 1 ended in exactly this: a high-roller customer database leaking out through a device nobody thought to secure.

Data leakage is often the goal, not the method — the other five threats are frequently just how an attacker gets there.

Connecting the Dots

~2 min

Each threat maps back to one control from Topic 2.

Device attacks	→	Device security
Eavesdropping	→	Encryption & secure comms
Spoofing	→	Authentication
DoS attacks	→	Availability
Malware	→	Device + network security
Data leakage	→	Confidentiality & cloud security

Every defense from Topic 2 has a real-world reason to exist — this is that reason, named.

The Unit IV roadmap

~1 min

6 topics, 180 minutes.



1 **Introduction**

2 **Security Fundamentals**

3 **Threats & Attacks**

You are here

4 **Data Analytics**

5 **Blockchain + IoT**

6 **Recap & Assessment**

You are here: Topic 3. Six numbered threats across 8 content slides — shorter than Topic 2, matching its 30 minutes.

The big picture

~2 min

Device attacks	Physical access to hardware — tampering, firmware extraction, cloned devices.
Eavesdropping	Passively listening to unencrypted traffic, then replaying what was captured.
Spoofing	Impersonating a device, gateway or location to gain false trust.
DoS attacks	Flooding, sleep deprivation, jamming, or a full DDoS — all targeting availability.
Malware	Scans, infects, spreads and acts — the pattern behind Mirai and similar botnets.
Data leakage	Sensitive data exposed via misconfiguration, weak APIs, or metadata.

One-line takeaway

“Every threat here has a name only because a control from Topic 2 was missing, weak, or skipped.”

Device attacks | Eavesdropping | Spoofing | DoS | Malware | Data leakage

Up next: Topic 4 — IoT Data Analytics: preprocessing, time-series, visualization, anomaly detection and predictive analytics.



Turning Raw Streams Into Insight

~3 min

A sensor produces numbers all day. Analytics is everything that happens between that number and a decision.

In Simple Words

A raw sensor reading is messy, timestamped and, on its own, almost meaningless. Analytics is the discipline of cleaning it, organizing it, and asking questions of it.

Some of today revisits ground from Unit III — anomaly detection and AI — but through a data-pipeline lens: how the numbers get there in the first place.

Today, in 45 minutes

The full path from a raw reading to a forecast a machine can act on.

Six ideas, in the order data flows

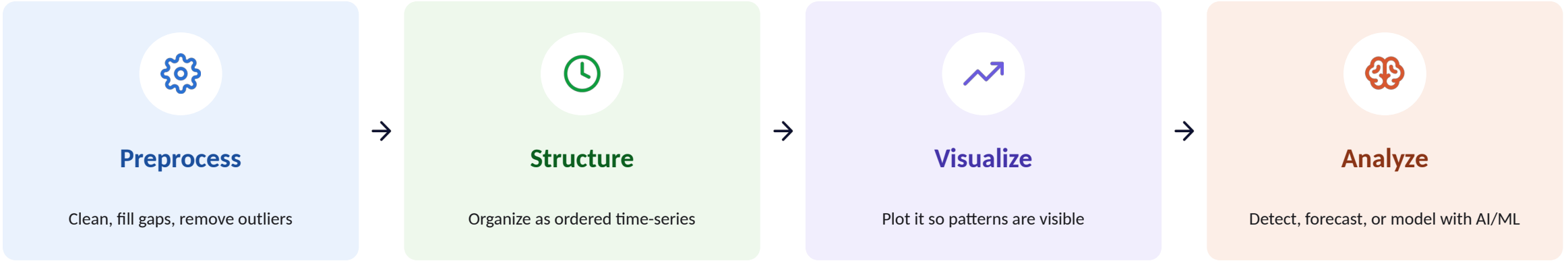
1. Preprocessing
2. Time-series data
3. Visualization
4. Anomaly detection
5. Predictive analytics
6. AI/ML analytics

Notice the order: clean the data before you plot it, plot it before you model it.

1 The Analytics Pipeline

~3 min

One straight line from a sensor reading to a decision.



Why this order matters

A model trained on dirty data learns the noise, not the pattern. A chart plotted before cleaning hides the real trend behind glitches. Every stage here depends on the one before it doing its job properly — which is exactly why the next four sections follow this same left-to-right order.

This is the same pipeline shape as Unit III's sensor-to-actuator walkthrough, aimed at insight instead of action.

2 Data Preprocessing — Why It’s Needed

~4 min

Raw sensor data is never as clean as a textbook example.

Missing readings

A sensor drops offline for a few minutes, leaving gaps in the sequence.

Noise

Electrical interference or a shaky connection adds small random errors.

Outliers

A momentary glitch produces one wildly wrong reading among thousands of good ones.

Inconsistent timing

Readings do not always arrive at perfectly even intervals.

None of this is a sign of a broken sensor — it is just what real-world data looks like before anyone touches it.

3 Data Preprocessing — Key Techniques

~4 min

A standard toolkit that fixes each problem from the last slide.

1 Cleaning

Remove or correct impossible values, like a negative temperature from a positive-only sensor

2 Filling gaps

Interpolate a missing reading from the values just before and after it

3 Normalization

Rescale different sensors (temperature, vibration) onto a comparable range

4 Outlier handling

Flag or smooth a reading far outside the expected range before it skews an analysis

Garbage in, garbage out

This is not a minor housekeeping step. An anomaly-detection model trained on noisy, unfiltered data will either miss real anomalies buried in the noise, or flag noise itself as an anomaly. Every technique that follows in this topic — visualization, forecasting, AI/ML — inherits whatever quality this stage leaves behind.

Data scientists often spend more time here than on any model that comes after it.

4 Time-Series Data

~4 min

What makes a sequence of timestamped readings its own kind of data.

The defining traits

Timestamped

Every value is tied to exactly when it was recorded

Ordered

The sequence itself carries information — shuffling it destroys meaning

Sampling rate

How often readings are taken shapes what patterns are even visible

Patterns you look for

Trend

A gradual rise or fall over a long stretch of time

Seasonality

A pattern that repeats on a schedule, like daily temperature cycles

Noise

Random variation left over once trend and seasonality are accounted for

This is the same "order matters" idea from Unit III's ML section, now the star of the show rather than a footnote.

5 Visualization

~4 min

The right chart turns a spreadsheet of numbers into an obvious pattern.



Line chart

The default for a single sensor over time — shows trend and seasonality at a glance.



Bar / histogram

Compares totals across categories, or shows how often values fall in each range.



Scatter plot

Reveals a relationship between two sensors, like vibration versus temperature.



Heatmap

Shows intensity across two dimensions at once, e.g. hour of day versus day of week.

A dashboard is just several of these charts, kept live and updating as new readings arrive.

Why this pipeline has to be automatic

~2 min

Millions

of readings per day, per site

even at one reading a second

Continuous

data, with no pause button

preprocessing has to keep up live

Thousands

of sensors, one dashboard

visualization has to scale too

Where anomaly detection, prediction and AI come in

At this scale, no person can watch every chart. The next three sections — anomaly detection, predictive analytics and AI/ML — are how the pipeline keeps working without a human staring at every dashboard: it watches the data for you and only raises a hand when something needs attention.

This is exactly why the pipeline ends in analytics, not just a chart.

6 Anomaly Detection — the Data-Analytics Lens

~3 min

Same idea as Unit III, seen from where the data comes from.

Statistical methods

Flag a reading a fixed distance from the mean, or outside a set range — simple, fast, works well on clean, preprocessed data.

ML-based methods

Learn a more complex notion of "normal" across several sensors at once — the Unit III approach, needs more data and preprocessing.

For the full treatment — how it works, real examples, predictive maintenance — see Unit III, Topic 5.

The technique has not changed since Unit III — only the reminder that it depends entirely on the pipeline built so far today.

7 Predictive Analytics

Forecasting a future value, not just flagging a past one.

~4 min

How it differs from anomaly detection

Anomaly detection asks "is this reading unusual, right now?" Predictive analytics asks a different question: "given the trend so far, what will the reading likely be next hour, next day, next week?"

What it needs

- Clean, well-structured time-series data
- Enough history to learn a trend or seasonal pattern
- A model — from a simple moving average to ML

Where it shows up

Predictive maintenance

Forecasting when a machine is likely to fail — Unit III's example of this exact idea

Demand forecasting

Predicting energy or water use ahead of time to plan supply

Capacity planning

Estimating when storage, bandwidth or compute will run out

Predictive maintenance from Unit III was one application of predictive analytics — this is the general idea behind it.

8 AI/ML-Based IoT Analytics

~4 min

Where all five ideas so far converge into one modeling layer.

Feeds on preprocessed data

Every model here is only as good as the cleaning and structuring done earlier today.

Learns from time-series

Patterns across time — trend, seasonality — become features a model can learn from.

Explained through visualization

A model's output is only useful once it is shown on a chart a person can read.

Powers detection & prediction

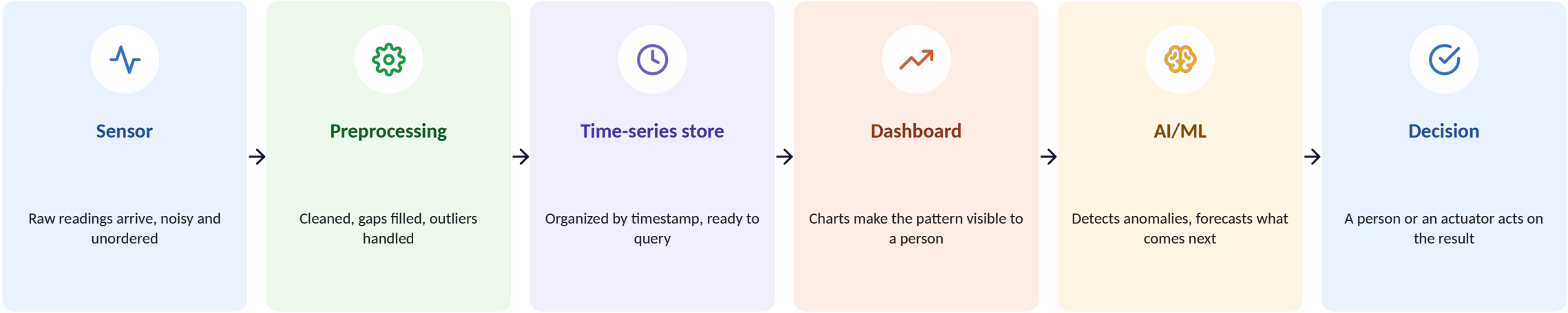
The same ML techniques run both the anomaly detector and the forecaster.

AI/ML is not a separate topic bolted on at the end — it is what the first five ideas were building toward all along.

The Full Pipeline, Walked Through

~4 min

Sensor → Preprocessing → Time-series store → Dashboard → AI/ML → Decision.



Reading it end to end

A vibration sensor's raw stream is cleaned and gap-filled, then stored as an ordered time-series. A dashboard turns it into a live chart an engineer can glance at. Underneath, an AI/ML model watches the same data continuously, flags anomalies and forecasts the trend — and when its confidence crosses a threshold, it triggers exactly the kind of maintenance decision covered in Unit III.

The Unit IV roadmap

~1 min

6 topics, 180 minutes — this topic's 45 minutes are the longest single block.



1 **Introduction**

2 **Security Fundamentals**

3 **Threats & Attacks**

4 **Data Analytics**

You are here

5 **Blockchain + IoT**

6 **Recap & Assessment**

You are here: Topic 4. Eight numbered sections across 11 content slides reflect this topic's 45 minutes.

By the end of this topic

You will be able to:



Explain

Why preprocessing has to happen before visualization or modeling, and what it fixes.



Identify

Trend, seasonality and noise in a time-series, and pick the right chart to show each.



Distinguish

Anomaly detection from predictive analytics, and describe how each depends on clean data.



Trace

The full pipeline from a raw reading to an AI/ML-driven decision.

Look at any IoT dashboard and ask: how many pipeline stages had to work for this chart to even exist?

The big picture

~2 min

Preprocessing	Clean, fill gaps, normalize and remove outliers — every later stage depends on this.
Time-series data	Timestamped, ordered readings with trend, seasonality and noise to look for.
Visualization	Line, bar, scatter and heatmap charts turn cleaned data into visible patterns.
Anomaly detection	Statistical or ML methods flag the unusual — full depth was Unit III, Topic 5.
Predictive analytics	Forecasts a future value from trend and history — predictive maintenance is one use.
AI/ML analytics	The modeling layer where preprocessing, time-series and visualization all converge.

One-line takeaway

> *“Every chart, alert and forecast an IoT system produces rests on data that was cleaned and organized first — that work is invisible, and it is everything.”*

Preprocessing | Time-series | Visualization | Anomaly detection | Predictive analytics | AI/ML

Up next: Topic 5 — Blockchain + IoT: decentralized IoT, smart contracts, data integrity and identity management.



Trust Without a Central Authority

~3 min

Every unit so far assumed one trusted cloud in the middle. This topic asks what happens without one.

In Simple Words

Every unit so far trusted one central cloud to authenticate devices, store data and run the analytics. That cloud is convenient, but it is also a single point of failure and a single point of trust.

Blockchain offers a different model: a shared, tamper-evident record that many parties keep together, with no single owner.

Today, in 35 minutes

The basics of blockchain, then four ways it meets IoT — and an honest look at the trade-offs.

Six ideas, building up

1. Why blockchain, why IoT
2. How blockchain works
3. Decentralized IoT
4. Smart contracts
5. Data integrity
6. Identity management

The first two slides are foundation — everything after them is blockchain applied specifically to IoT.

1 Why Blockchain, Why IoT?

~4 min

The problem a central cloud creates, and what blockchain offers instead.

The centralized problem

Single point of failure

If the cloud goes down, every device loses its source of truth

One trusted party

Every device has to trust the cloud operator completely

Hard to audit

Records can be quietly changed by whoever controls the server

What blockchain offers

No single owner

Many participants keep a copy of the same record together

Tamper-evident

Changing a past record breaks the chain in a way everyone can see

Shared, not siloed

Multiple organizations can trust the same data without a middleman

Not every IoT system needs this — the next slide shows how blockchain actually works before we judge when it helps.

2 How Blockchain Works

~4 min

A quick primer, in four ideas, before applying it to IoT.



Block

A batch of records, bundled together with a timestamp.



Chain

Each block stores the previous block's hash, linking them in order.



Distributed ledger

Every participant keeps a full copy of the entire chain.



Consensus

Participants agree on the next valid block before it is added.



Why changing history is so hard

Editing an old block changes its hash, which breaks every block linked after it — visible immediately to anyone holding a copy of the chain.

This is the same hash-linking idea from Topic 2's encryption, now used to protect a history instead of a message.

3 Decentralized IoT

~4 min

Devices coordinate with each other, without one server orchestrating everything.

The shift in architecture

Every architecture in this course so far routed device-to-device interactions through a gateway or cloud. Decentralized IoT lets devices transact directly, recording the interaction on a shared ledger instead of a private database.

What it enables

- Devices trading resources directly, e.g. spare bandwidth or energy
- No single gateway that, if compromised, controls everything
- A shared, verifiable log every participant can check

Centralized vs Decentralized

	CLOUD	BLOCKCHAIN
Trust	One operator	The network itself
Failure mode	One outage stops all	Keeps running elsewhere
Speed	Fast, one hop	Slower, needs consensus
Best for	Most IoT systems	Multi-party, no-trust cases

Decentralization is a trade-off, not a strict upgrade — the last row of this table is the whole point.

4 Smart Contracts

~4 min

Code on the blockchain that runs itself when a condition is met.

Condition set

A rule is written into the contract in advance, e.g. "if temperature > 8°C"

→

Sensor reports

An IoT device sends a reading that the contract is watching for

→

Condition checked

The blockchain network verifies the condition against the rule

→

Contract executes

Automatically, with no human or company needing to approve it

A worked example: the cold-chain shipment

A pharmaceutical shipment has a smart contract that releases payment only if a temperature sensor never reports above 8°C for the entire trip. If the cold chain is broken even once, the contract withholds payment automatically — no invoice, no dispute, no trusted middleman required to make the call.

A smart contract is only as trustworthy as the sensor feeding it — which is why data integrity comes next.

Where this pairing already shows up

~2 min

Supply chain

Tracking a product from factory to shelf on a shared, tamper-evident ledger

Energy trading

Households with solar panels selling surplus power directly to neighbors

Insurance

Sensor-verified claims that pay out automatically via smart contract

Notice what these have in common

Each example involves multiple organizations that do not fully trust each other, yet need to agree on the same facts. That combination — many parties, low mutual trust, verifiable IoT data — is exactly the situation where blockchain earns its overhead. A single company with its own devices and its own cloud rarely needs it.

Keep that test in mind for the trade-offs slide coming up.

5 Data Integrity

~4 min

Proving a sensor reading has not been altered since it was recorded.

Hash the reading

Every sensor reading is fingerprinted with a cryptographic hash the moment it is captured.

Write it to the chain

That hash, not the raw data, is recorded on the blockchain alongside a timestamp.

Verify anytime, later

Anyone can re-hash the original data and compare it to the stored hash to detect tampering.

Why this beats a normal database

A database administrator can quietly edit a row. Rewriting a hash already chained into the blockchain would break every block after it — visible to every participant holding a copy.

This is exactly what makes a smart contract from the previous section trustworthy in the first place.

6 IoT Identity Management

Giving every device its own verifiable identity, without a central registry.

~4 min

The problem it solves

Topic 2's authentication assumed one company issues and manages every device certificate. That breaks down across billions of devices from many manufacturers, especially once devices change owners or cross organizational lines.

How blockchain helps

- Each device holds its own key pair as its identity
- That identity is registered once, on a shared ledger
- Any participant can verify it, with no single issuer

Life of a device identity

1. **Manufactured**

   A key pair is generated and burned into the hardware

2. **Registered**

   The public key is written to the blockchain as its identity

3. **Verified, anywhere**

   Any network it joins later can check that identity independently

This is authentication from Topic 2, reimagined without one company controlling the registry.

The Honest Trade-offs

~3 min

Blockchain is powerful, but it is not free — especially for small devices.

Compute & power cost

Running consensus and verifying blocks is far heavier than a tiny sensor can usually handle.

Latency

Waiting for consensus can take seconds — too slow for the real-time control from Unit III.

Storage growth

The ledger only grows; a constrained device cannot hold a full, ever-expanding copy.

The practical pattern

Most real deployments keep the heavy device itself off-chain, running Unit III's edge and cloud pipeline as normal, and only write a hash or a critical event to the blockchain — getting tamper-evidence where it matters most, without asking a sensor to run full consensus.

Use the "many parties, low trust" test from earlier — if that is not true, blockchain is probably overkill.

The Unit IV roadmap

~1 min

6 topics, 180 minutes.



1. **Introduction**
2. **Security Fundamentals**
3. **Threats & Attacks**
4. **Data Analytics**
5. **Blockchain + IoT** — You are here
6. **Recap & Assessment**

You are here: Topic 5. Six numbered sections across 10 content slides reflect this topic's 35 minutes.

By the end of this topic

You will be able to:



Explain

How a blockchain's block, chain, ledger and consensus work together.



Describe

How a smart contract executes automatically from an IoT sensor condition.



Justify

When blockchain-based data integrity and identity are worth their overhead.



Weigh

The power, latency and storage trade-offs against Unit III's constrained devices.

Look at any multi-party IoT system and ask: do these parties actually distrust each other enough to need this?

The big picture

~2 min

Why blockchain	Removes the single point of failure and single trusted party a central cloud requires.
How it works	Blocks, chained by hash, replicated across a distributed ledger, agreed on by consensus.
Decentralized IoT	Devices transact directly and log it on a shared ledger, instead of through one gateway.
Smart contracts	Code that executes automatically once an IoT condition is verified — no middleman needed.
Data integrity & identity	Hashed readings and device identities that anyone can verify, with no single issuer.
Up next	Topic 6: Recap & Self-Assessment — security checklist, analytics pipeline recap, and a preview of Unit V.

One-line takeaway

> *“Blockchain trades speed and simplicity for trust without a middleman — worth it exactly when many parties need to agree, and not much sooner.”*

Blockchain basics | Decentralized IoT | Smart contracts | Data integrity | Identity management

Up next: Topic 6 — Recap & Self-Assessment: security checklist, analytics pipeline recap, and a preview of Unit V.



Closing the Loop on Unit IV

~2 min

No new material — this topic checks what stuck across the last five topics before Unit V begins.

1 Introduction

2 Security Fundamentals

3 Threats & Attacks

4 Data Analytics

5 Blockchain + IoT

Three things this recap does

Runs the security checklist

The eight fundamentals from Topic 2 and how the six threats from Topic 3 break them.

Retraces the analytics pipeline

Preprocessing through AI/ML, in the order data actually flows, from Topic 4.

Checks your understanding

Five quick questions covering Topics 1 through 5, then a look ahead to Unit V.

Twenty focused minutes — five topics, one thread: security depends on data, data depends on trust.

1 Security Fundamentals, Checked Off

~3 min

The eight ideas from Topic 2, as a quick-reference list.

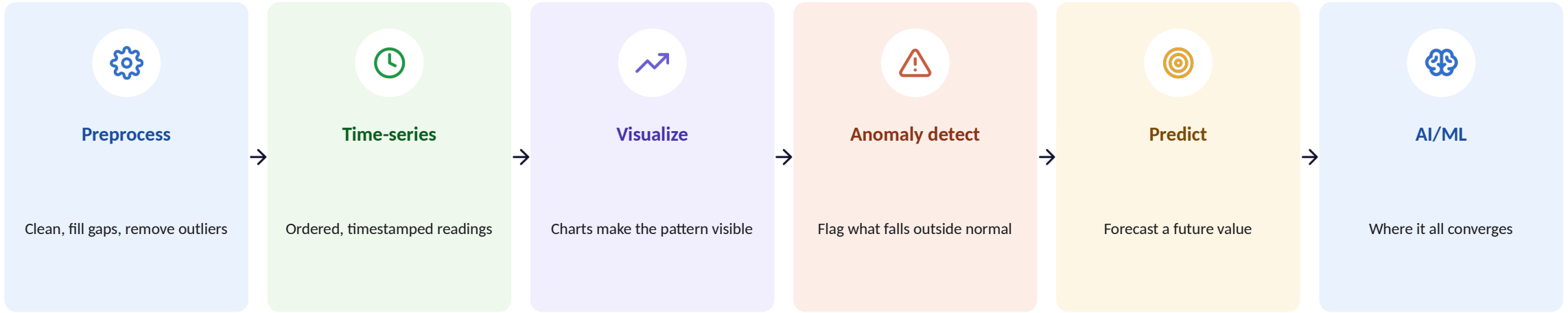
Confidentiality, Integrity, Availability	The classic CIA triad — what "secure" starts with
Authenticity & Non-repudiation	IoT's two extra requirements, for devices that act on their own
Authentication	Proving a device or user really is who it claims to be
Authorization	Deciding what a verified identity is actually allowed to do
Encryption	Symmetric for speed, asymmetric to safely share the key
Secure communication	The TLS/DTLS handshake that protects data on the move
Device, network & cloud security	The same requirements, applied at every layer — defense in depth

Every threat in Topic 3 — device attacks to data leakage — broke exactly one of these seven rows.

2 The Analytics Pipeline, Retraced

~3 min

Six stages from Topic 4, in the order data actually flows.



The rule that ran through all of it

Clean the data before you plot it, plot it before you model it. Anomaly detection, prediction and AI/ML all inherit whatever quality preprocessing and structuring left behind.

Full depth on anomaly detection lives back in Unit III, Topic 5 — this is the data-pipeline half of that story.

Self-assessment: five quick questions

~4 min

1. Which security requirement does a Denial-of-Service attack target most directly?
2. What is the difference between authentication and authorization, in one sentence?
3. Why must data be preprocessed before it is visualized or modeled?
4. What is the difference between anomaly detection and predictive analytics?
5. When is blockchain worth its overhead for an IoT system, and when is it not?

Answers are not printed here on purpose — talk them through with a partner before checking Topics 2–5.

Unit IV, the big picture

~2 min

Introduction	Real breaches — Mirai, the Jeep hack, the fish-tank thermometer — framed why security matters.
Security Fundamentals	Requirements, authentication, authorization, encryption, secure comms, defense in depth.
Threats & Attacks	Device attacks, eavesdropping, spoofing, DoS, malware, data leakage — each breaks one control.
Data Analytics	Preprocessing, time-series, visualization, anomaly detection, prediction, AI/ML.
Blockchain + IoT	Decentralized trust, smart contracts, data integrity, identity — and their real overhead.
This topic	Security checklist, analytics recap, self-assessment, and a look at Unit V.

Up Next: Unit V

~2 min

IoT Applications — 180 minutes, seeing the same pipeline reused across seven real domains.

1	2	3	4	5	6	7	8
Introduction	Smart Agriculture	Healthcare IoT	Industrial IoT	Smart City	Energy & Transport	Smart Home	Wrap-Up & Future
10 min	25 min	25 min	25 min	25 min	30 min	20 min	20 min

Every domain in Unit V reuses the same sensor-to-decision pipeline — just aimed at a different real-world problem.

One-line takeaway

> *"Security keeps the data trustworthy; analytics turns trustworthy data into insight — Unit IV was about making both hold up under real conditions."*

Security fundamentals | Threats & attacks | Data analytics | Blockchain + IoT | Recap & assessment

Up next: Unit V — IoT Applications, starting with how every domain reuses the same sensor-to-action pipeline.

